COLLABORATIVE FAUTL TOLERANCE IN FOG COMPUTING USING REVERSE AUCTION AND GOSSIP

Abdus Saboor Gaffari
Department of Computer Science and Engineering
College of Engineering
American University of Sharjah
Sharjah, UAE
b00105302@aus.edu

Solomon Ghebretatios
Department of Computer Science and Engineering
College of Engineering
American University of Sharjah
Sharjah, UAE
b00104390@aus.edu

Affan Murtaza
Department of Computer Science and Engineering
College of Engineering
American University of Sharjah
Sharjah, UAE
b00104949@aus.edu

December 2, 2025

ABSTRACT

Suspendisse vitae elit. Aliquam arcu neque, ornare in, ullamcorper quis, commodo eu, libero. Fusce sagittis erat at erat tristique mollis. Maecenas sapien libero, molestie et, lobortis in, sodales eget, dui. Morbi ultrices rutrum lorem. Nam elementum ullamcorper leo. Morbi dui. Aliquam sagittis. Nunc placerat. Pellentesque tristique sodales est. Maecenas imperdiet lacinia velit. Cras non urna. Morbi eros pede, suscipit ac, varius vel, egestas non, eros. Praesent malesuada, diam id pretium elementum, eros sem dictum tortor, vel consectetuer odio sem sed wisi.

1 Introduction

2 Related Work

Fault tolerance in distributed IoT execution environments has been approached from several directions, including container migration, proactive failure prediction, and fault-aware scheduling at the fog and edge layers. Together, these works emphasize the importance of maintaining service continuity under dynamic and resource-constrained conditions, while also revealing that integrated migration-centric mechanisms remain relatively under-explored.

Early works on edge–cloud container migration have focused primarily on improving migration decision-making. [**?**] proposed a fuzzy logic-based container migration mechanism for IoT-driven edge–cloud systems. The study considers CPU, memory, and network conditions to guide migration decisions, highlighting the value of adaptive, multi-metric decision models for reliability in heterogeneous environments. The framework relies on container runtimes that support CRIU (Checkpoint/Restore In Userspace), a widely used Linux utility for capturing a container's in-memory state and restoring it elsewhere.However, The study proposes the conceptual migrations process and lacks end-to-end implementation.

A more comprehensive container-level mechanisms is proposed in other Kubernetes-focused research which aim to incorporate off-the-shelf readily available technologies. [**?**] introduces a proactive architecture that combines BiLSTM based fault prediction with a stateful migration mechanism integrated into the Kubernetes control plane. Their use of CRIU enables transferring in-memory boot and execution states and demonstrates how stateful recovery can reduce

startup delays and improve availability. This work establishes the feasibility of performing full state transfer for containerized services and shows its utility for maintaining service continuity.

Within fog computing, fault tolerance has mostly been considered at the task or module scheduling level. [**?**] proposed a Markov-based reliability model that dynamically allocates workloads to stable fog nodes. [**?**] similarly integrate reliability considerations into a hybrid scheduling approach that improves load distribution and success rates for latency-sensitive IoT tasks. These approaches offer useful models for assessing fog-layer reliability, although their mitigation actions occur through task reassignment rather than full service migration.

The work in [**?**] proposed the ECLB framework, which applies Bayesian classification and checkpointing to manage load and improve reliability in fog-based IoT applications. Their proposed model incorporates energy-aware backup placement and dynamic balancing of modules across fog devices. The framework illustrates how lightweight statistical methods can strengthen task continuity. Similarly authors in [**?**] also proposes a task based decentralized allocation strategy with the aim of achieving resilience through group formation and local cooperation during node failures.

The authors in [**?**] proposed two complementary frameworks for decentralized task offloading in vehicular edge environments. In the first framework, called the Autonomous Vehicular Edge (AVE), each vehicle gathers lightweight beacons from nearby peers and applies an ant colony optimization strategy to offload tasks based on neighbor availability, link quality, and mobility characteristics. The second framework, Hybrid Vehicular Cloud (HVC), extends this model by incorporating roadside units and cloud servers, and uses an online scheduler to decide dynamically whether a task should be executed by a neighboring vehicle, an RSU, or the cloud. Both frameworks demonstrate the benefits of decentralized decision-making and multi-tier offloading paths for reducing latency and maintaining high availability.

Moreover, surveys such as [**?**] have attempted to summarize architectural and scheduling techniques ranging from reinforcement learning to stochastic optimization, which are used to improve fault tolerance in fog computing systems. These reviews highlight the constant challenges related to dynamic systems, heterogeneity of resources, and real-time reliability in fog environments which point toward the need for unified solution solutions that offer orchestration and recovery across distributed compute tiers.

Other works have also explored mechanisms for enforcing truthfulness in collaborative distributed computing paradigms. [**?**] explores reverse-auction–based resource allocation models in cloud settings. In their work, users bid for computational resources and the system allocates them based on fair pricing and optimal social welfare. The framework attempts to ensure allocation outcomes remain both efficient and economically balanced. While it does not consider fault tolerance, the study underscores the relevance of fairness, cost-awareness, and multi-tenant optimization for guiding allocation and migration decisions. Lastly, [**?**] proposed an online auction-based incentive mechanism for Collaborative Edge Computing, where edge servers independently decide whether to accept incoming tasks based on dynamic virtual pricing of CPU, storage, and bandwidth. The framework uses primal–dual optimization to handle stochastic task arrivals, soft deadlines, and heterogeneous valuations while ensuring truthfulness and competitive social welfare.The study illustrates how decentralized edge servers can make autonomous, locally informed decisions in distributed environments.

Across these works, several gaps remain evident. Existing fog and container-fault-tolerance mechanisms often rely on centralized schedulers, where migration or task-placement decisions are made by a single broker or cluster manager as seen in [**?**][**?**][**?**][**?**]. These centralized designs can unintentionally force fog nodes to accept migrated tasks or containers, even when local conditions make them unsuitable. Moreover, many fault-tolerant fog approaches operate strictly at the task level rather than supporting stateful container mobility, while predictive mechanisms such as those used by [**?**] and [**?**] do not integrate migration execution into a single, continuous workflow. Taken together, the literature shows limited support for decentralized, locally informed migration decisions in which fog nodes themselves retain control over accepting, rejecting, or proposing alternative placement options based on real-time resource conditions. This is especially crucial in a multi-owner environment such as that proposed in our approach. Our study addresses these gaps by designing a proactive fault-tolerance mechanism in which each fog node participates directly in a collaborative decentralized decision-making, and evaluates migration candidacy using its own state, neighbourhood conditions, and partial-view system context. We aim to design a framework which avoids rigid top-down placement policies, allowing nodes to maintain autonomy, and enable container migration to proceed in a manner that better reflects the volatility, dynamic agent behaviour, and resource heterogeneity inherent in real-world fog computing environments.

3 Methodology / Materials and Methods

3.1 Proposed Approach

The proposed framework introduces a collaborative fault-tolerant model for fog computing in which autonomous fog nodes cooperate to maintain service continuity when failures are predicted or detected. The system operates

within a multi-tier edge–fog–cloud hierarchy, where fog nodes act as the primary execution layer and collaborate only when necessary, without relying on any centralized controller or forced offloading. Collaboration emerges through decentralized, voluntary interactions between independently administered fog nodes, enabling both proactive FT through early migration and reactive FT when faults occur unexpectedly.

Two core mechanisms underpin the collaborative behavior. First, a gossip-based state dissemination protocol allows each fog node to gradually build an eventually complete and consistent view of the global resource state. By exchanging periodic summaries of CPU, memory, and bandwidth loads across a lightweight logical topology, fog nodes gain situational awareness without broadcasting costly global messages. This state is not required to be perfectly synchronized; instead, it serves as a sufficiently accurate substrate for decentralized decision-making during fault recovery. Second, a reverse-auction mechanism enables multi-owner fog nodes to negotiate container placements when local recovery is not possible. A node experiencing a predicted or actual fault initiates an auction in which candidate fog nodes respond with bids reflecting their capacity and estimated migration cost. Collaborative FT emerges because nodes voluntarily participate, compete, and are compensated for accepting foreign containers, unlike forced or centrally dictated offloading.

The fault-tolerance workflow operates through a concise three-stage migration strategy: (i) intra-fog migration attempts to relocate containers within the same fog node whenever a healthy server is available; (ii) if local recovery is infeasible, inter-fog migration is triggered, where candidate fog nodes are evaluated using gossip-informed scoring and reverse-auction bidding; and (iii) if no suitable collaborator exists, the task is escalated to the cloud as a final fallback.

The framework executes tasks in lightweight, migratable containers to allow seamless pausing, checkpointing, transfer, and resumption across servers or fog nodes. This ensures that when a fault is predicted, all containers on the affected server are immediately checkpointed and migrated. If an unexpected fault occurs, the resulting behavior depends on the fault type: CPU and bandwidth degradations still permit checkpointing, whereas crashes cause complete loss of progress, requiring immediate restart elsewhere.

By combining container-based execution, gossip-driven distributed awareness, and reverse-auction–based negotiation, the proposed system provides a decentralized, prediction-aware, and economically aligned FT architecture suitable for heterogeneous, multi-owner fog environments.

3.2 System Model

The system consists of three interconnected layers: edge devices, fog nodes, and a cloud layer that provides a global but slower and more resource-intensive fallback. As shown in the architectural model, each fog node hosts multiple heterogeneous servers managed by a local control layer and an FT predictor module. The fog layer forms the main execution substrate, processing tasks originating from stationary edge devices. These devices continuously generate indivisible tasks that must run to completion and therefore cannot be split or distributed across servers. Each task executes inside a dedicated container, and each server can host multiple such containers concurrently, constrained only by its CPU, RAM, and bandwidth capacities. Containers can be paused, checkpointed, migrated, and resumed across any fog server or cloud resource, allowing mobility in response to failures.

The system assumes a multi-owner fog environment, where each fog node is controlled by an independent administrator. Nodes may strategically misreport internal metrics such as CPU load, memory usage, bandwidth availability, or expected migration time. However, they cannot fake observable outcomes, including actual migration duration, SLA adherence, or execution-time overload. This distinction is essential: misreporting affects negotiation, but actual behavior determines accountability. The framework embeds this assumption directly into its FT logic rather than treating it as an external constraint.

Tasks arriving at a fog node are placed onto the best available server based on that node's own local scheduling policy. The proposed FT framework does not modify this placement algorithm; instead, it activates only when faults occur or are predicted. The FT predictor is assumed to exist and guarantees perfect predictions: no false positives or false negatives. Thus, every predicted fault will occur, and any server not flagged by the predictor is considered safe until a new prediction arises.

Three types of faults are considered: CPU degradation, bandwidth degradation, and server crash. While these faults differ in operational impact, the FT model treats them uniformly in terms of triggering migration, except for one distinction: crashes render servers completely inaccessible, causing absolute loss of container progress, whereas degradations allow checkpointing before migration. The fog-to-cloud communication path exhibits significantly higher latency and lower bandwidth than inter-fog communication links, reinforcing the preference for local or inter-fog recovery before escalating to the cloud.

This system model enables decentralized interaction among fog nodes, preserves each operator's autonomy, supports truthful or untruthful behavior within defined limits, and leverages containerization to provide strong fault-migration guarantees across the hierarchy.

3.3 Gossip Protocol

As the proposed collaborative FT system does not rely on any central coordinator for maintaining global visibility we adopt a lightweight and scalable mechanism for disseminating resource availability information among fog nodes. The framework uses an eventual consistency gossip protocol that lets each fog node keep an approximate, but still fairly current view of the system wide resource availability without relying on full broadcast. As the environment involves multi owner fog nodes, this mechanism allows the information exchange to stay light and decentralized, while still providing enough shared visibility for early FT decisions. This eventually consistent state information also allows the fog nodes to make earlier decisions, since even an approximate and slightly stale view can still guide quick migration choices before a fault fully materializes.

In this mechanism every node maintains a table with its knowledge of the whole system's state, called $StateTable_i$. To keep the network usage to a minimum we adopt a ring topology for the gossip dissemination through the system. In this topology, every fog node has a designated fog node that it receives an update from and a designated fog node that it sends the update to. Each fog node periodically snapshots its internal load conditions using the normalized CPU, memory, and bandwidth loads given by the below equations:

$$L^i_{cpu} = \frac{\sum_{s \in S_i} U^s_{cpu}}{\sum_{s \in S_i} C^s_{cpu}} \tag{1}$$

$$L^i_{mem} = \frac{\sum_{s \in S_i} U^s_{mem}}{\sum_{s \in S_i} C^s_{mem}} \tag{2}$$

$$L^i_{bw} = \frac{\sum_{s \in S_i} U^s_{bw}}{\sum_{s \in S_i} C^s_{bw}} \tag{3}$$

These load values are then attached with a timestamp and updated in its $StateTable_i$, then this updated $StateTable$ is forwarded to its designated upstream ring neighbor. Whenever a fog node receives a $StateTable$ update from its designated downstream ring neighbor, it updates its local $StateTable_i$ with all the updated states. Algorithm 1 presents the complete gossip dissemination process executed periodically at each fog node.

Since fog nodes belong to independent owners, the proposed system assumes that nodes can misreport their loads during gossip exchanges. A node may share lower load values to look more available and attract incoming containers, and this is treated as a normal possibility in such an environment. The architecture is designed so that this kind of misreporting becomes noticeable later through the trust scoring process, where SLA results and actual migration behavior expose the gaps between what was claimed and what actually happened. The proposed approach also assumes that the gossip mechanism is implemented in a simple way where nodes only update their own local values when forwarding the table to the next node, which keeps the process predictable in a multi owner setting. In this way, the gossip process works as the decentralized base that the proposed collaborative FT system relies on. It does not force honesty, but it provides enough shared visibility for the reverse auction and the trust scoring parts to work properly.

3.4 Reverse Auction for Collaborative Migration

The proposed approach adopts a reverse auction, similar to [**?**, **?**], to handle inter fog migration in a way that fits the proposed collaborative system model. Since no node can be forced to take additional load, the system needs a negotiation process rather than a central assignment or distributed forced migration similar to [**?**, **?**, **?**], especially in a multi owner environment where each node follows its own local preferences and constraints. The reverse auction provides a mechanism which allows the candidate nodes to state their willingness to host an incoming container and describe the cost they expect if they take it. This creates a collaborative environment where nodes offer bids that reflect their current conditions, and the originating node can pick a destination that seems reasonable given the information it receives.

When a predicted or detected fault cannot be recovered by local migration on a fog node, the originating fog node issues a migration request to selected candidates determined via gossip-based scoring, discussed later. When a candidate fog node receives a migration request from the originating fog node, it evaluates the request based on the migration time

Algorithm 1 Gossip State Dissemination at Fog Node i

Input: Set of servers S_i, state table $StateTable_i$, neighbor j (upstream), current time t
Output: Updated state tables at neighboring fog nodes

// Compute local load snapshot
$activeServers \leftarrow 0$
$L^i_{cpu} \leftarrow 0$, $L^i_{mem} \leftarrow 0$, $L^i_{bw} \leftarrow 0$
for each server $s \in S_i$ **do**
  if s is not faulted and not predicted to fail **then**
    $L^i_{cpu} \leftarrow L^i_{cpu} + \frac{U^s_{cpu}}{C^s_{cpu}}$, $L^i_{mem} \leftarrow L^i_{mem} + \frac{U^s_{mem}}{C^s_{mem}}$, $L^i_{bw} \leftarrow L^i_{bw} + \frac{U^s_{bw}}{C^s_{bw}}$
    $activeServers \leftarrow activeServers + 1$
  end if
end for
if $activeServers > 0$ **then**
  $L^i_{cpu} \leftarrow L^i_{cpu}/activeServers$, $L^i_{mem} \leftarrow L^i_{mem}/activeServers$, $L^i_{bw} \leftarrow L^i_{bw}/activeServers$
else
  $L^i_{cpu} \leftarrow 1$, $L^i_{mem} \leftarrow 1$, $L^i_{bw} \leftarrow 1$
end if

// Create local state entry with timestamp
$entry_i \leftarrow$ GossipStateEntry$(L^i_{cpu}, L^i_{mem}, L^i_{bw}, t)$
$StateTable_i[i] \leftarrow entry_i$

// Propagate state table to ring neighbors
Send $StateTable_i$ to fog node j

// Upon receiving state table $StateTable_k$ from fog node k (downstream):
for each entry $(nodeId, state) \in StateTable_k$ **do**
  if $nodeId \notin StateTable_i$ or $state.timestamp > StateTable_i[nodeId].timestamp$ **then**
    $StateTable_i[nodeId] \leftarrow state$
  end if
end for

and the impact that it expects on itself. This is formulated as the Bid_{value} that this fog node sends to the originating node. This Bid_{value} is essentially the cost that the fog node at the receiving end of the migration will incur, and is calculated as:

$$Bid_{value} = T^{claimed}_{mig} + k \cdot Impact \tag{4}$$

Here $T^{claimed}_{mig}$ is the total migration time that the node expects based on its expectation of bandwidth it can provide, $Claimed_{bw}$, the time to pause the container at the originating node, t_{pause}, and the time to resume the container at the destination, t_{resume}.

$$T^{claimed}_{mig} = t_{pause} + \frac{S_{container}}{Claimed_{bw}/8} + t_{resume} \tag{5}$$

The second component of the bid measures the impact the node might experience by hosting the container based on the below equation:

$$Impact = \frac{R_{cpu}}{C_{cpu}} + \frac{R_{mem}}{C_{mem}} + \frac{R_{bw}}{Claimed_{bw}} \tag{6}$$

This formulation allows the migration overhead and the local resource load to be represented in one simple cost value, so a fog node that is already under heavy load will produce a higher bid and signal that it should not take any more load. Algorithm 2 shows how each candidate fog node builds its bid based on the formulation discussed. After all the individual bids are received, the origin fog node needs to evaluate these bids. During this evaluation the node also incorporates the trust value, from the trust mechanism discussed in next section, within the bids to handle cases where a

node may not report its status honestly, either in the bid or the gossip. This results in the effective bid to be evaluated as:

$$Bid_{eff}(i) = \frac{Bid_{value}(i)}{\max(\varepsilon, t(i))} \tag{7}$$

where $t(i)$ is the trust score for node i, discussed in detail in next section. This simple formulation makes nodes with lower trust look more expensive, so they are less likely to be selected even if their raw bid is low. Over time this pushes dishonest behaviour out of the system because nodes that keep misreporting slowly lose competitiveness, while reliable ones become the natural winners. The origin fog node always selects the bidding node with the lowest Bid_{eff}. The full bid evaluation and winner selection logic at the origin fog node is given in Algorithm 3.

Algorithm 2 Bid Calculation at Candidate Fog Node i

Input: Migration request for container c from origin fog node j
Output: Bid response containing feasibility and bid value

// Retrieve container requirements
$R_{cpu} \leftarrow c.demandMips$
$R_{mem} \leftarrow c.ramMb$
$R_{bw} \leftarrow c.bandwidth$
$S_{container} \leftarrow c.containerSizeMb$
$D_{sla} \leftarrow c.deadlineSeconds$

// Compute claimed migration time
$Claimed_{bw} \leftarrow$ available bandwidth to fog node j
$T_{transfer} \leftarrow S_{container}/(Claimed_{bw}/8)$
$T_{mig}^{claimed} \leftarrow t_{pause} + T_{transfer} + t_{resume}$

// Compute projected resource load
$L_{cpu} \leftarrow$ current CPU load at fog node i
$L_{mem} \leftarrow$ current memory load at fog node i
$L_{bw} \leftarrow$ current bandwidth load at fog node i
$projected_{cpu} \leftarrow L_{cpu} + R_{cpu}/C_{cpu}$
$projected_{mem} \leftarrow L_{mem} + R_{mem}/C_{mem}$
$projected_{bw} \leftarrow L_{bw} + R_{bw}/Claimed_{bw}$

// Check feasibility
$timeToDeadline \leftarrow \max(0, D_{sla} - t)$
$hasHealthyServer \leftarrow$ any server at i is not faulted and not predicted to fail
$feasible \leftarrow hasHealthyServer$ and $projected_{cpu} \leq 1$ and $projected_{mem} \leq 1$
and $projected_{bw} \leq 1$ and $T_{mig}^{claimed} \leq timeToDeadline$

// Compute bid value
$Impact \leftarrow \frac{R_{cpu}}{C_{cpu}} + \frac{R_{mem}}{C_{mem}} + \frac{R_{bw}}{Claimed_{bw}}$
$Bid_{value} \leftarrow T_{mig}^{claimed} + k \cdot Impact$ ▷ k is resource impact weight

return BidResponse$(i, c.id, feasible, Bid_{value}, Claimed_{bw}, T_{mig}^{claimed})$

3.5 Trust Scoring and Reputation Evolution

Because the proposed collaborative FT mechanism assumes that fog nodes are independently owned and may behave strategically, i,e. lie about their resources, a robust trust scoring process is required to maintain truthful behavior across the gossip exchanges and bidding. For this purpose we propose a trust scoring mechanism where every fog node maintains a trust value $t(i) \in [0, 1]$ for every other fog node in the system. This trust score maintains each node's reliability based on past interactions and is updated after every completed task using both the migration and execution results. This mechanism is essential in enabling collaboration without imposing a central coordinator.

The trust mechanism evaluates every winning node's honesty by comparing the *claimed* versus *actual* migration performance. The *claimed* bandwidth, $Claimed_{bw}$, is shared by the bidding node in the bid response, and the actual bandwidth, $Actual_{bw}$, can be calculated from the observed transfer behavior using:

Algorithm 3 Bid Evaluation and Winner Selection at Origin Fog Node

Input: Container c, set of bid responses $Bids$, trust scores T, trust threshold θ_{trust}
Output: Selected winner fog node or cloud fallback

$FeasibleBids \leftarrow \emptyset$
for each bid $b \in Bids$ **do**
  $bidderId \leftarrow b.bidderId$
  $t(bidderId) \leftarrow T[bidderId]$

  // Filter by trust threshold
  if $t(bidderId) < \theta_{trust}$ **then**
    continue ▷ Exclude low-trust nodes
  end if

  // Verify feasibility using local knowledge
  if not $b.feasible$ **then**
    continue
  end if

  // Compute effective bid using trust adjustment
  $Bid_{eff}(bidderId) \leftarrow \frac{b.bidValue}{\max(\varepsilon, t(bidderId))}$ ▷ $t(i)$ is the trust value of node i

  // Compute projected headroom for tie-breaking
  $headroom(bidderId) \leftarrow (1 - projected_{cpu}) + (1 - projected_{mem}) + (1 - projected_{bw})$

  $FeasibleBids \leftarrow FeasibleBids \cup \{(bidderId, Bid_{eff}, headroom, latency)\}$
end for

// Select winner using lexicographic ordering
if $FeasibleBids \neq \emptyset$ **then**
  Sort $FeasibleBids$ by (Bid_{eff} ascending, $headroom$ descending, $latency$ ascending)
  $winner \leftarrow$ first element in sorted $FeasibleBids$
  return $winner.bidderId$
else
  return cloud fallback
end if

$$Actual_{bw} = \frac{S_{container}}{T_{transfer}} \times 8 \tag{8}$$

and thus the relative bandwidth honesty error, e_{bw}, can be determined by (ε is to avoid division by 0):

$$e_{bw} = \frac{|Claimed_{bw} - Actual_{bw}|}{\max(\varepsilon, Claimed_{bw})} \tag{9}$$

Similarly, the discrepancy between claimed migration time , $T_{mig}^{claimed}$, and actual migration times T_{mig}^{actual} can be evaluated through:

$$T_{mig}^{actual} = t_{pause} + T_{transfer} + t_{resume} \tag{10}$$

where $T_{transfer}$ is the actual transfer time observed by the origin node. The relative migration time honesty error, e_{mig}, can be determined by:

$$e_{mig} = \frac{|T_{mig}^{claimed} - T_{mig}^{actual}|}{\max(\varepsilon, T_{mig}^{claimed})} \tag{11}$$

We define the parameters τ_{bw} and τ_{mig} that represent the dishonesty thresholds for bandwidth and migration time, respectively. Error values above these thresholds indicate intentional misreporting. In case misreporting of migration

time and bandwidth is observed, i.e. $e_{bw} > \tau_{bw}$ or $e_{mig} > \tau_{mig}$, the trust update applies a multiplicative decay to the trust score using:

$$t'(i) = t(i) \cdot \delta_{decay} \tag{12}$$

where δ_{decay} is a configurable trust decay value.

Finally, to detect dishonest execution related metrics such as understated CPU or memory load, the framework evaluates SLA performance using:

$$m_{sla} = \frac{D_{sla} - T_{complete}}{\max(\varepsilon, D_{sla})} \tag{13}$$

where m_{sla} is the SLA margin for the migrated task and negative margins indicate SLA violations, and $T_{complete}$ is the actual completion time of the task. When $m_{sla} < -\tau_{sla}$, where τ_{sla} defines the tolerance for SLA deviation, the trust score is penalized again using the below equation:

$$t''(i) = \max(0, t'(i) - \beta_{violation}) \tag{14}$$

where, $\beta_{violation}$ quantifies how strongly such violations reduce trust.

On the contrary, the proposed trust model also rewards consistently honest fog nodes through:

$$t''(i) = \min(1, t'(i) + \beta_{success}) \tag{15}$$

whenever all honesty conditions are satisfied. The parameter $\beta_{success}$ governs the incremental trust gained from compliant behavior.

Therefore, the final trust value, in case of lying or honest node, is given as:

$$t(i) = t''(i) \tag{16}$$

which integrates both migration-stage honesty checks and SLA-based execution verification. The two phase update to trust score is because migration-stage dishonesty require a multiplicative decay, whereas SLA violations use an additive penalty since an SLA miss may arise from genuine overload rather than deliberate lying. Separating these updates allows the system to distinguish intentional misreporting from operational performance issues while still reflecting both in the final trust value. The parameters τ_{bw}, τ_{mig}, and τ_{sla} therefore define the boundaries of acceptable error, while $\beta_{violation}$ and $\beta_{success}$ control how penalties and rewards shape long-term trust evolution. Together, these mechanisms ensure that repeated dishonesty, whether through gossip, bidding, or execution, reduces a node's influence in future auctions, while sustained honest behavior is economically reinforced.

In addition to these active updates, trust also improves passively over time so that nodes that have not recently hosted tasks can gradually regain competitiveness. The recovery magnitude is proportional to the remaining distance to perfect trust i.e. 1, and at periodic intervals the system applies

$$t_{passive}(i, t) = \min\left(1,\ t(i) + r_{passive} \cdot (1 - t(i))\right) \tag{17}$$

where $r_{passive}$ is a small recovery rate applied at regular time intervals. This slow rise prevents long-term exclusion of inactive nodes while ensuring that previously dishonest nodes do not regain trust abruptly. The complete trust update mechanism, integrating both active evaluation and passive recovery, is formalized in Algorithm 4.

3.6 Migration Workflow

The proposed migration workflow provides a structured way for the system to decide where a container should move when its current execution can no longer continue due to a predicted fault. The workflow integrates the gossip protocol, reverse-auction mechanism, and trust scoring discussed so far into one decision path that the system follows whenever a migration is required. It defines a three-phase approach that first attempts local recovery, then moves to collaborative inter-fog migration, and finally uses the cloud only when no other option is feasible. Algorithm 6 outlines the full decision process. When the fault predictor identifies that a server is approaching failure, all containers running on that server are paused, checkpointed if possible and removed from that server. Once containers are removed from the failing server, the system initiates the migration workflow.

Algorithm 4 Trust Evaluation for Node i and Passive Recovery

Input: Fog node i, container c, actual metrics $(Actual_{bw}, T_{mig}^{actual})$, claimed metrics $(Claimed_{bw}, T_{mig}^{claimed})$, completion time $T_{complete}$
Output: Updated trust score $t(i)$

// Phase 1: Migration and bandwidth honesty evaluation
$e_{bw} \leftarrow \frac{|Claimed_{bw} - Actual_{bw}|}{\max(\varepsilon, Claimed_{bw})}$
$e_{mig} \leftarrow \frac{|T_{mig}^{claimed} - T_{mig}^{actual}|}{\max(\varepsilon, T_{mig}^{claimed})}$

$t'(i) \leftarrow t(i)$ ▷ Initialize with current trust
if $e_{bw} > \tau_{bw}$ or $e_{mig} > \tau_{mig}$ **then**
    $t'(i) \leftarrow t(i) \cdot \delta_{decay}$ ▷ Apply multiplicative decay
end if

// Phase 2: SLA-based evaluation
$m_{sla} \leftarrow \frac{D_{sla} - T_{complete}}{\max(\varepsilon, D_{sla})}$

if $m_{sla} < -\tau_{sla}$ **then**
    $t''(i) \leftarrow \max(0, t'(i) - \beta_{violation})$ ▷ Apply additive penalty
else
    if $e_{bw} \leq \tau_{bw}$ and $e_{mig} \leq \tau_{mig}$ **then**
        $t''(i) \leftarrow \min(1, t'(i) + \beta_{success})$ ▷ Apply additive reward
    else
        $t''(i) \leftarrow t'(i)$ ▷ No SLA reward if migration dishonest
    end if
end if

$t(i) \leftarrow t''(i)$ ▷ Store updated trust

// Passive recovery (applied periodically for all fog nodes)
procedure PASSIVERECOVERY($i, t_{current}, t_{last\_recovery}$)
    if $t_{current} - t_{last\_recovery} \geq \Delta t_{recovery}$ **then**
        $increment \leftarrow r_{passive} \cdot (1 - t(i))$
        $t(i) \leftarrow \min(1, t(i) + increment)$
        Update $t_{last\_recovery}(i) \leftarrow t_{current}$
    end if
end procedure

return $t(i)$

3.6.1 Phase 1: Intra-fog Recovery

The workflow first attempts to keep the container within the same fog node. In order to do that, the local scheduler checks whether another healthy server within the fog node is available to host the container. If such a server exists, the container resumes execution locally and the migration completes within the fog node. If no server can accommodate the container, the FT system escalates to the next phase and initiates collaborative evaluation across other fog nodes.

3.6.2 Phase 2: Inter-fog migration via reverse auction

When the container cannot be migrated within the same fog node, the FT system attempts collaborative FT. In this phase the workflow evaluates the collaborating fog nodes using the state information that was exchanged through the gossip mechanism. First the nodes with stale state entries are filtered out. Then for every remaining candidate node, the FT system computes the workload proportions based on the container being migrated,

$$p_{cpu}, \qquad p_{mem}, \qquad p_{bw}, \tag{18}$$

where p_{cpu}, p_{mem}, and p_{bw} represent how much of the container's demand is CPU-intensive, memory-intensive, or bandwidth-intensive. These proportions describe the container's dominant resource profile and always sum to one.

The system then determines the task urgency through the normalized slack factor,

$$U_{norm} = 1 - \frac{\max(0,\, D_{sla} - T_{pred})}{D_{sla}}, \tag{19}$$

where D_{sla} is the task's deadline and T_{pred} is the predicted remaining execution time. Larger U_{norm} values indicate that the deadline is approaching and the task becomes more sensitive to delays.

These values produce the dynamic scoring weights. The bandwidth, CPU and memory weight are given by

$$\gamma = p_{bw} + U_{norm}(1 - p_{bw}), \tag{20}$$

$$\alpha = (1-\gamma)\frac{p_{cpu}}{p_{cpu} + p_{mem}}, \qquad \beta = (1-\gamma)\frac{p_{mem}}{p_{cpu} + p_{mem}}, \tag{21}$$

where γ increases when the workload is bandwidth-heavy or when urgency is high, and α and β split the remaining weight proportionally based on the CPU versus memory composition of the container.

Each candidate fog node i is then assigned a dynamic score based on Eq. 22:

$$Score(i) = \alpha\left(1 - L^i_{cpu}\right) + \beta\left(1 - L^i_{mem}\right) + \gamma\left(1 - L^i_{bw}\right), \tag{22}$$

where L^i_{cpu}, L^i_{mem}, and L^i_{bw} are the normalized CPU, memory, and bandwidth loads of node i taken from gossip. The $(1 - L)$ terms indicate the available remaining capacity in each resource dimension.The complete scoring procedure is formalized in Algorithm 5

Candidates that achieve high scores proceed to the negotiation stage. The FT system then initiates a reverse auction, with these candidate nodes, where the origin fog node transmits the container details to the candidate nodes. Each candidate fog node submits a bid calculated using Eq. 4 and sends them to the origin node as the bid response (which also includes other information). The origin node then transforms these submitted bids into effective bids by applying the trust adjustment rule according to Eq. 7, and the fog node with lowest effective bid is selected (As this is a reverse auction). The container is then migrated to the selected fog node, and the observed transfer time T_{transfer} is recorded. On arrival, the receiving fog node places the container on one of its healthy servers using its internal scheduler.

If no collaborating fog node is able to host the container, the workflow moves to the cloud fallback phase.

3.6.3 Phase 3: Cloud Fallback

When neither local nor inter-fog recovery is possible, the workflow sends the container to the cloud as the final fallback option. The cloud does not participate in the bidding process. The container is transferred, T_{transfer} is recorded, and the container resumes or restarts depending on whether a checkpoint was available.

Overall, the workflow creates a structured and flexible response to failures. It prioritizes local recovery, leverages distributed scoring and auction-based cooperation when needed, and uses the cloud only as a final fallback. This layered approach maintains task continuity across both predicted and unexpected fault scenarios, even in environments where fog nodes might misreport their state or behave strategically.

3.6.4 Payment Settlement

Once the migrated task completes, the node that won the auction and executed the container informs the origin node about the completion. The origin fog node on receiving this alert evaluates the final payment by applying the final payment rule defined in Algorithm 7, and carries out the payment to the winning node.

The final payment to the winner node is carried out with a penalty attached to it for missing the SLA or reporting untruthful capacities. The final payment amount is determined using the below equation:

$$\text{Payment} = \max\left(0,\; Bid_{Value} - P_{\text{lie}} - P_{\text{sla}}\right), \tag{23}$$

where the P_{lie} is the dishonesty penalty for lying about the bandwidth and migration time and P_{sla} is the penalty for SLA violation.

The dishonesty penalty is calculated as

$$P_{\text{lie}} = \lambda\left(e_{bw} + e_{mig}\right), \tag{24}$$

Algorithm 5 Dynamic Scoring of Fog Nodes for Container Migration

Input: Container c, state table $StateTable$, staleness threshold θ_{stale}, current time t
Output: Ranked list of candidate fog nodes

// Compute workload proportions
$total \leftarrow R_{cpu} + R_{mem} + R_{bw}$
$p_{cpu} \leftarrow R_{cpu}/total$
$p_{mem} \leftarrow R_{mem}/total$
$p_{bw} \leftarrow R_{bw}/total$

// Compute normalized urgency
$slack \leftarrow \max(\varepsilon, D_{sla} - t)$
$U \leftarrow 1/slack$
$U_{norm} \leftarrow \min(1.0, U \cdot k)$ ▷ $k = 0.1$ is urgency scaling factor

// Compute dynamic weights
$\gamma \leftarrow p_{bw} + U_{norm}(1 - p_{bw})$
$W_{remaining} \leftarrow 1 - \gamma$
$\alpha \leftarrow W_{remaining} \cdot \frac{p_{cpu}}{p_{cpu}+p_{mem}}$
$\beta \leftarrow W_{remaining} \cdot \frac{p_{mem}}{p_{cpu}+p_{mem}}$

// Score all non-stale fog nodes
$Candidates \leftarrow \emptyset$
for each fog node i in $StateTable$ **do**
    if i is local node **then**
        continue
    end if
    $entry \leftarrow StateTable[i]$
    if $t - entry.timestamp > \theta_{stale}$ **then**
        continue ▷ Discard stale entries
    end if
    $Score(i) \leftarrow \alpha(1 - L^i_{cpu}) + \beta(1 - L^i_{mem}) + \gamma(1 - L^i_{bw})$
    $Candidates \leftarrow Candidates \cup \{(i, Score(i))\}$
end for

// Sort by score descending
Sort $Candidates$ by score in descending order
return $Candidates$

where e_{bw} and e_{mig} are derived from trust scoring using Eq. 9 and Eq. 11 and λ controls the severity of this penalty.

The SLA penalty is calculated as

$$P_{\text{sla}} = \mu \cdot \max\left(0,\ -m_{sla}\right). \tag{25}$$

Here m_{sla} is the SLA margin of the completed task from the trust score using Eq. 13, and μ scales how strongly SLA violations reduce the final payment.

Intra-fog placements always yield zero payment, and cloud placements follow the same rule while not affecting trust values. This settlement step finalizes the workflow by aligning the economic incentives of the hosting fog nodes with the honesty and performance expectations defined earlier.

3.7 Effectiveness of Solution

This section explains how the proposed system behaves when fog nodes try to behave strategically or dishonestly. The explanations are written as different scenarios that walk through what happens in each case. The reasoning still comes from the mechanisms already described earlier, such as the bidding formulation, the gossip scoring, the trust update equations with the two-phase adjustments, and the SLA-based evaluation that happens naturally after execution. The goal here is not to introduce anything new, but just to show how the pieces already introduced end up supporting the system even when nodes do not behave nicely.

Algorithm 6 Collaborative Migration Workflow at Fog Node i

Input: Container c, fault event f, servers S_i, state table $StateTable_i$
Output: Container migrated to new location

// Handle fault and checkpoint if possible
$sourceServer \leftarrow$ server hosting container c
if f is irrecoverable **then**
    Remove c from $sourceServer$ without checkpointing
    $c.resetProgress()$ ▷ Work progress lost, restart required
else
    Pause container c
    Checkpoint c with current progress at time t ▷ Fault is recoverable
    Remove c from $sourceServer$
end if

// Phase 1: Attempt intra-fog migration
$localTarget \leftarrow$ default scheduler places c on a healthy server in S_i
if $localTarget \neq$ null **then**
    Resume execution of c
    Record intra-fog migration
    return
end if

// Phase 2: Inter-fog migration via reverse auction
$Candidates \leftarrow$ Score all fog nodes using Algorithm 5
Limit $Candidates$ to top N_{max} nodes

// Request bids from candidates
$Bids \leftarrow \emptyset$
for each fog node $j \in Candidates$ **do**
    Send migration request to j
    Await bid response from j (computed via Algorithm 2)
    $Bids \leftarrow Bids \cup \{$ received bid$\}$
end for

// Select winner using trust-adjusted bidding
$winner \leftarrow$ SelectWinner$(c, Bids)$ using Algorithm 3
if $winner \neq$ null and $winner \neq$ cloud **then**
    Initiate transfer of c to fog node $winner$
    Measure actual bandwidth $Actual_{bw}$ and migration time T_{mig}^{actual}
    Winner's default scheduler places c on appropriate server
    Record inter-fog migration
    return
end if

// Phase 3: Cloud fallback
if cloud is available **then**
    Initiate transfer of c to cloud
    Cloud's default scheduler places and resumes/restarts c
    Record cloud migration
else
    Enqueue c for retry when resources become available
end if

Scenario 1: A fog node understates its bid. If a fog node tries to understate its bid by claiming a smaller migration time or a lower resource impact than what it actually has, then it might look attractive at first. It may even win a bid occasionally because the origin node simply sees a lower cost. But as soon as the container finishes migration, the actual bandwidth and the actual migration duration are measured using the equations already

Algorithm 7 Payment Settlement and Combined Trust Update

Input: Container c, winning fog node i, completion time $T_{complete}$, bid context
Output: Payment transfer and updated trust score

// Retrieve bid and actual metrics
$Bid_{value} \leftarrow$ original bid value from fog node i
$Claimed_{bw} \leftarrow$ bandwidth claimed in bid
$T_{mig}^{claimed} \leftarrow$ migration time claimed in bid
$Actual_{bw} \leftarrow$ measured actual bandwidth during transfer
$T_{mig}^{actual} \leftarrow$ measured actual migration time

// Compute error metrics
$e_{bw} \leftarrow \frac{|Claimed_{bw} - Actual_{bw}|}{\max(\varepsilon, Claimed_{bw})}$, $e_{mig} \leftarrow \frac{|T_{mig}^{claimed} - T_{mig}^{actual}|}{\max(\varepsilon, T_{mig}^{claimed})}$

// Compute SLA margin
$m_{sla} \leftarrow \frac{D_{sla} - T_{complete}}{\max(\varepsilon, D_{sla})}$

// Update trust using combined evaluation (Algorithm 4)
$t_{updated}(i) \leftarrow$ TrustUpdate$(i, c, Actual_{bw}, T_{mig}^{actual}, Claimed_{bw}, T_{mig}^{claimed}, T_{complete})$

// Calculate payment with penalties
$P_{lie} \leftarrow \lambda(e_{bw} + e_{mig})$, $P_{sla} \leftarrow \mu \cdot \max(0, -m_{sla})$

$Payment \leftarrow \max(0, Bid_{value} - P_{lie} - P_{sla})$

// Execute payment transfer
if fog node $i \neq$ owner of c and $i \neq$ cloud **then**
    Debit $Payment$ from owner's account
    Credit $Payment$ to fog node i's account
end if

Record settlement metrics: $(c.id, i, Bid_{value}, Payment, t_{updated}(i), m_{sla})$

given for B_{actual} and the real $T_{\text{mig,actual}}$. When the claimed numbers and the actual numbers do not match, the relative errors appear, and these errors immediately activate the multiplicative trust decay in Phase 1 of the trust update. And once that happens, the trust value decreases and the node's future effective bids become much higher because effective bids divide by trust. So the node that understates its bid ends up becoming less competitive later. This means the short-term gain is lost quickly, and in many cases it becomes much worse off. So understating a bid does not really help the node, and the system naturally pushes it out.

Scenario 2: A fog node overstates its bid. If a fog node overstates its bid and reports a much higher cost than necessary, the system does not punish it with trust decay because the actual measurements still match what was claimed. But this behavior still does not work out well for the node. The inflated bid simply places it at the bottom of the winner selection, and because winner selection always tries to reduce the effective bid, such a node almost never gets chosen. In some sense, the node excludes itself by being too expensive. This means overstating does not give any advantage, and it slowly pushes the node away from being part of the collaboration. The system does not need to punish it, because the bidding logic already does that indirectly.

Scenario 3: A fog node lies about bandwidth or migration time. A fog node may try to lie by putting nicer claimed bandwidth or shorter claimed migration times into its bid. But once the migration actually takes place, the real transfer bandwidth and the real migration duration can be computed directly from the observed transfer using the earlier equations. These measured values go into the relative error formulas, and when the discrepancy becomes noticeable, the thresholds for dishonesty are crossed. As soon as that happens the trust update does the multiplicative penalty, and sometimes the additive one too if the SLA is also harmed. And because trust affects the effective bid so strongly, even a few dishonest bids make the node almost impossible to select in future migrations. So lying about bandwidth or migration time is always detected eventually because we do not rely on the claimed values, we rely on what actually happened.

Scenario 4: A fog node lies about CPU or RAM load. CPU and memory dishonesty is a bit harder because these values cannot be checked directly during the bidding process. But they show up later through the SLA margin. If a fog node lies and says it has plenty of available CPU or RAM and then tries to run the container with less than promised, the execution slows down. The actual completion time becomes larger, and the SLA margin becomes negative, and this automatically triggers the additive trust penalty in Phase 2. If this continues to happen multiple times, the trust value keeps dropping. Once trust goes below the threshold, the node stops being considered at all. So even though the system does not directly check CPU or RAM claims, it relies on the SLA margin to expose dishonesty. And since SLA violations are observed facts, a fog node cannot hide the results of its lies.

Scenario 5: A fog node lies in the gossip state. A fog node may try to cheat by reporting a lighter load in the gossip state, hoping to look more attractive when scores are calculated. But gossip load values come from actual server measurements, so this type of lying is already constrained. And even if the node could somehow lower its reported load, it still has to execute the containers that it attracts. If it is actually overloaded, execution slows down and that leads to SLA violations. And these SLA violations directly activate the additive penalty in the trust update. So even if gossip lying helped in the short term, the resulting poor performance would appear very quickly, and trust would decrease. And since lower trust makes bidding worse, the node eventually stops being selected anyway. So lying in gossip does not really help, and eventually it becomes counterproductive.

Scenario 6: A fog node uses multiple dishonest strategies together. A fog node may think that if it mixes different types of lies, like fake gossip values, wrong CPU load reports, underestimated migration times, or overstated bandwidth, then maybe the system cannot catch all of it. But because the trust update works in two separate phases, and because SLA outcomes are always observable, the penalties accumulate. Migration dishonesty activates the multiplicative decay in Phase 1. SLA dishonesty activates the additive penalty in Phase 2. When both happen together, the trust falls even faster. Once trust becomes small, the effective bid becomes extremely large, and the node almost never gets selected. So combining dishonest behaviors does not hide anything; instead it accelerates exclusion from the collaboration.

These scenarios show, in a practical and scenario-driven way, that the mechanisms introduced earlier work together so that lying or misreporting is not a sustainable strategy. The system keeps moving toward nodes that behave honestly, because honest nodes naturally get better trust, better effective bids, and fewer penalties. The result is that the proposed FT system remains stable and cooperative even when nodes try to game the process.

3.8 Experimental Setup

The evaluation of the proposed collaborative FT approach was conducted using a customized version of iFogSim2[1] that we modified to support all components of our design, including container-level execution, checkpointing, migration, gossip-driven state dissemination, urgency-aware scoring, reverse-auction bidding, and trust-based negotiation. These extensions allowed the simulator to faithfully reproduce the behavior of the proposed mechanisms rather than relying on the default iFogSim model. All workloads, failures, and communication events were generated synthetically within the modified simulator, enabling controlled experiments and full reproducibility without reliance on external datasets.

Table 1 summarizes the system-level configuration used in all experiments. The simulated environment consists of ten fog nodes arranged in a logical ring for gossip communication. Each fog node hosts three to four heterogeneous servers with capacities in the range of $C^s_{cpu} \in [2000, 2200]$ MIPS, $C^s_{mem} = 4096$ MB, and $C^s_{bw} \in [3500, 4500]$ Mbps. Each fog node is connected to six stationary edge devices that continuously generate tasks. Cloud resources were provisioned with significantly higher capacity to serve as a reliable fallback destination. Network delays follow the fog-computing hierarchy, where edge–fog links are fastest, inter-fog communication is moderately fast, and fog–cloud links are slower and more bandwidth-constrained.

Table 2 summarizes the faults, workload parameters, bidding configuration, trust parameters, and SLA specifications used in the experiment. Faults are generated according to a Poisson distribution. The faults are injected and arrive with a mean inter-failure time of 60 seconds, and each fault includes a prediction window of 300 seconds and a recovery time of 240 seconds. Three types of faults are considered namely: CPU degradation, bandwidth degradation, and server crash, which follow the probabilities listed in the Table 2. CPU and bandwidth degradations allow the system to checkpoint containers before migrating them, while server crashes cause complete loss of progress. During inter-fog migration, fog nodes respond with bids computed using the claimed migration time and resource impact formulations defined in Equations 5 and 6, parameterized by the constants k, t_{pause}, and t_{resume}. Trust scores evolve according to measured discrepancies in bandwidth and migration honesty, SLA behavior, and passive recovery, governed by the thresholds τ_{bw}, τ_{mig}, and τ_{sla}.

[1] https://github.com/Cloudslab/iFogSim

Task workloads were selected to produce realistic load conditions across the fog layer. Each edge device generates 20 tasks with exponential inter-arrival times, using configurable jitter. Resource requirements for tasks including CPU, memory, bandwidth, runtime, and container size, are sampled from the value ranges shown in Table 2. SLA deadlines scale with task runtime according to the multiplier provided in the configuration.

A baseline system was implemented for comparison, identical to the proposed mode except that gossip dissemination, trust scoring, and auction-based collaboration were disabled. Instead, all migration decisions were handled by a centralized scheduler. The centralized scheduler has realtime global view of all servers capacities and it can force fog nodes to take on containers that it wants to migrate. This allowed direct comparison between our decentralized collaborative FT approach and a simplified centralized system under identical workload and fault conditions.

Table 1: System and Network Configuration

Parameter	**Value**
Number of Fog Nodes	10
Servers per Fog Node	3–4
CPU Capacity per Server (C^s_{cpu})	2000–2200 MIPS
Memory per Server (C^s_{mem})	4096 MB
Bandwidth per Server (C^s_{bw})	3500–4500 Mbps
Storage per Server	100000 MB
Uplink/Downlink Bandwidth	6000–9000 Mbps
Cloud CPU Capacity	9000 MIPS
Cloud Memory	45000 MB
Cloud Bandwidth	40000 Mbps
Cloud Storage	200000 MB
Cloud Uplink/Downlink	40000 Mbps
Edge Devices per Fog Node	6
Edge Latency	3 ms
Edge Bandwidth	1500 Mbps
Edge Heterogeneity Jitter	10%
Inter-Fog Latency	8 ms
Inter-Fog Bandwidth	1500 Mbps
Fog-Cloud Latency	70 ms
Fog-Cloud Bandwidth	150 Mbps
Gossip Enabled	True
Gossip Interval	7 s
Gossip Staleness Threshold	3 intervals

3.9 Metrics for Comparison

To understand how the proposed collaborative FT approach behaves compared to the baseline configuration, we rely on a set of metrics that capture how well the system maintains timeliness, how smoothly it handles migrations, and how stable it remains when faults occur. These metrics highlight the practical influence of gossip visibility, trust-aware bidding, and prediction-driven migration, and they help show how these components shape the flow of tasks through the fog layer.

SLA violation rate. Each task in the system is associated with a completion deadline, and a violation is recorded whenever the task finishes after this deadline. Since both predicted faults and unexpected degradations can delay execution or increase migration overhead, the SLA violation rate provides a direct indication of whether the proposed FT framework improves reliability under fault conditions. A lower violation rate typically reflects more stable execution.

Average migration time. This measures how long a container remains in the migration process, beginning at the pause event and ending when execution resumes at the destination. A container may experience several migrations during its lifetime, especially in environments with repeated or overlapping faults, and for this reason average migration time becomes an important metric to observe. It shows whether the FT mechanisms keep recovery overhead manageable or whether repeated migrations accumulate into noticeable performance penalties. Lower values usually indicate smoother and more predictable recovery.

Table 2: Fault, Bidding, Trust, and Task Configuration

Category	Parameter and Value
Fault Model	
Prediction Lead Time	300 s
Mean Time Between Failures	60 s
Fault Start Delay	30 s
Recovery Time	240 s
CPU Failure Probability	0.33
Bandwidth Degradation Probability	0.34
Server Crash Probability	0.33
Bidding Model	
Max Fog Bidders	4
Pause Time (t_{pause})	1.5 s
Resume Time (t_{resume})	1.5 s
Resource Impact Weight (k)	0.6
Trust Model	
Trust Enabled	True
Decay Factor (δ_{decay})	0.5
Trust Recovery Factor	0.05
Trust Threshold	0.5
Bandwidth Dishonesty Threshold (τ_{bw})	0.3
Migration Dishonesty Threshold (τ_{mig})	0.5
SLA Violation Threshold (τ_{sla})	0.3
SLA Success Bonus ($\beta_{success}$)	0.02
SLA Violation Penalty ($\beta_{violation}$)	0.10
Payment Dishonesty Penalty (λ)	0.25
Payment SLA Penalty (μ)	0.10
Passive Recovery Rate ($r_{passive}$)	0.01
Passive Recovery Interval	21 s
Task Model	
Tasks per Edge Device	20
Mean Inter-Arrival Time	160 s
Runtime Range	240–400 s
Demand MIPS Range	200–600 MIPS
RAM Range	128–512 MB
Bandwidth Range	150–550 Mbps
Container Size Range	200–420 MB
SLA Runtime Multiplier	1.2

Average makespan time. Makespan represents the total duration a task spends in the system, including execution time, any delays caused by predicted or reactive faults, and all migration-related overhead. We compute this value for each task and then average across the workload. Makespan offers a broad view of end-to-end performance and helps reveal whether tasks are progressing efficiently despite migrations and disruptions.

Total number of migrations. To better understand how the system responds to faults, we track the total number of migrations and classify them into intra-fog, inter-fog, and cloud migrations. Intra-fog migrations reflect the system's ability to recover locally. Inter-fog migrations show how often cooperation among fog nodes is required. Cloud migrations indicate situations where neither local recovery nor inter-fog support is feasible. Observing the distribution of these migration types helps compare the behavior of the proposed FT approach with the baseline configuration, particularly in terms of how often unnecessary or avoidable migrations occur.

Together, these metrics provide a comprehensive view of responsiveness, efficiency, and stability, allowing us to compare the proposed FT approach with the baseline configuration in a controlled and meaningful way.